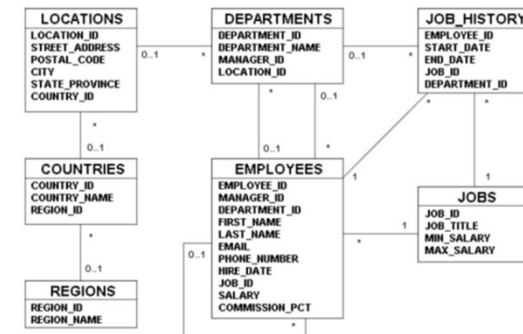


Chapitre 1 : Rappels et compléments SQL

Préambule

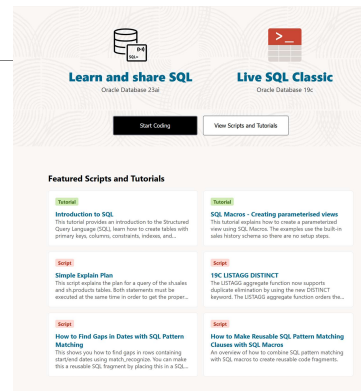


LiveSQL Oracle

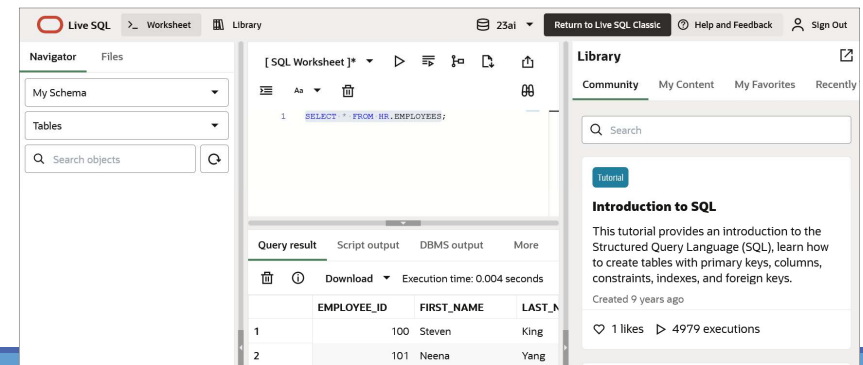
Créer un compte Oracle

Accès à LiveSQL

- pour coder en SQL rapidement
- Accéder à des exemples et documentation



LiveSQL Oracle

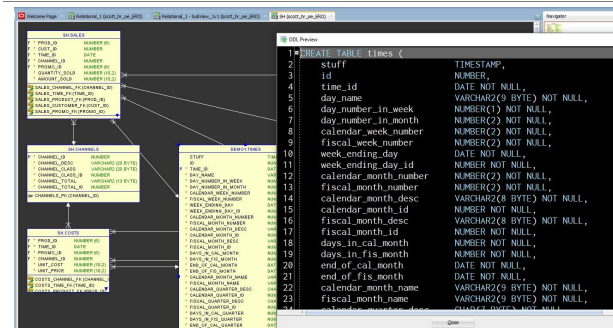


Oracle SQL Developer



6

Exemple Oracle



7

Ligne de commande



8

Modélisation

Des champs peuvent être « compliqués »

Exemple : un nom versus un pseudonyme

9

Modélisation

Importance du choix des entités

Signifiantes

Exemples du schéma HR Oracle

–Employees

–Departments

10

Environnement de modélisation

Interaction

Equipe projet

Modélisation de l'univers métier / société

Mauvaise modélisation → Mauvaise base !

◦ Non indexation des clés étrangères → pb

Le modèle de données : source de vérité

11

Conception et Modèles

Logique

Relationnel

Physique

13

Fichiers d'un projet

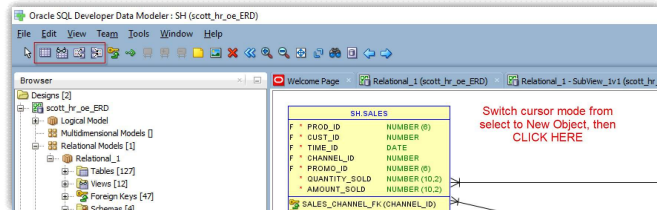
Création : fichier .DMD

Il existe un répertoire Design

Tout objet a un fichier

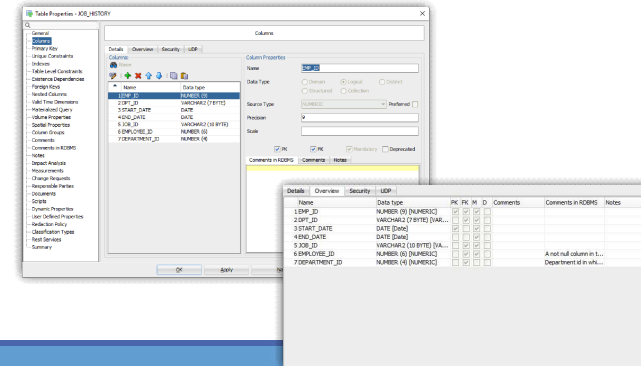
14

Définition Entité et Table



15

Propriétés



16

Clés naturelles ou de substitution

Attributs du monde réel

De substitution : chaîne de caractères ou nombres générés par le SGBD

17

Question ...

<https://app.wooclap.com/OZSURO>

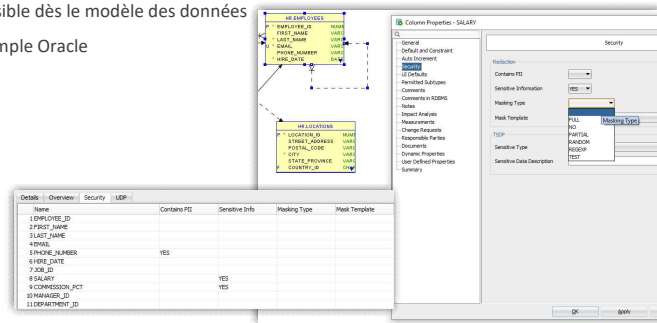
Le numéro de Sécurité Sociale est-il une bonne clé primaire en général ?

18

Sécurité des données

Possible dès le modèle des données

Exemple Oracle



19

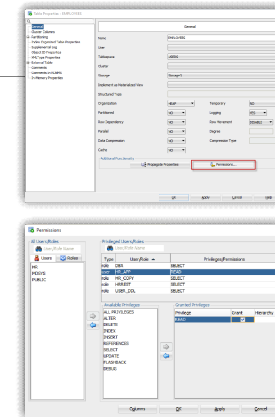
Sécurité dans la BD

SGBD permet la protection des données

Importance des rôles

- Admin / User ...
- Accès à certains objets

Exemples Oracle



21

Question

<https://app.wooclap.com/HLXXIQ>

Les clés étrangères dans le cadre de la modélisation d'une BD sont sources de mauvaise performance ?

- Vrai
- Faux
- Cela dépend des circonstances

22

Notation Barker utilisée par Oracle

Suit le modèle Entité / Association (ou E/R)

Pour une entité :

- Rectangle arrondi avec le nom de l'entité
- Chaque attribut est accompagné de sa signification :
 - * pour obligatoire,
 - o pour facultatif
 - et # pour UID

Une relation

- Ne peut exister qu'entre deux entités au maximum
- Peut s'appliquer sur une même entité (dite récursive)

23

Traitement des relations

Notation Barker / Oracle	Notation MERISE
1 A doit être relié à 1 B 1 B doit être relié à 1 A	1 A est relié à 1 et 1 seul B 1 B est relié à 1 et 1 seul A
1 A peut être relié à 0 ou 1 B 1 B peut être relié à 0 ou 1 A	1 A est relié à 0 ou 1 B 1 B est relié à 0 ou 1 A

24

Traitement des relations

Notation Barker / Oracle	Notation MERISE
1 A peut être relié à 1 B 1 B doit être relié à 1 A	1 A est relié à 0 ou 1 B 1 B est relié à 1 et 1 seul A
1 A doit être relié à 1 ou plusieurs (n) B 1 B doit être relié à 1 A	1 A est relié à 1 à n B 1 B est relié à 1 et 1 seul A
1 A peut être relié à 0 ou plusieurs B 1 B peut être relié à 0 ou 1 A	1 A est relié à 0 à n B à 0 ou 1 A est relié 1 B
1 A peut être relié à 0 ou plusieurs B 1 B doit être relié à 1 A	1 A est relié à 0 à n B 1 B est relié à 1 et 1 seul A

25

Traitement des relations

Notation Barker / Oracle	Notation MERISE
1 A peut être relié à 0 ou plusieurs B B hérite de l'identifiant de A	1 A est relié à 0 à n B La relation est identifiante pour B
1 A doit être relié à 1 à n B 1 B peut être relié à 0 ou 1 A	1 A est relié à 1 à n B 1 B est relié à 0 ou 1 A
1 A doit être relié à 1 ou plusieurs (n) B 1 B doit être relié à 1 ou plusieurs (n) A	1 A est relié à 1 à n B 1 B est relié à 1 à n A

26

Traitement des relations

Notation Barker / Oracle	Notation MERISE
1 A peut être relié à 0 ou plusieurs (n) B 1 B peut être relié à 0 ou plusieurs (n) A	1 A est relié à 0 à n B 1 B est relié à 0 à n A
1 A doit être relié à 1 ou plusieurs (n) B 1 B peut être relié à 0 ou plusieurs (n) A	1 A est relié à 1 à n B 1 B est relié à 0 à n A
1 A peut être relié à 0 ou plusieurs (n) B 1 B doit être relié à 1 ou plusieurs (n) A	1 A est relié à 0 à n B 1 B est relié à 1 à n A

27

Reverse engineering

Possible avec l'outil Oracle après connexion à la base

Importer un dictionnaire de données

28

Collaboratif

Subversion / Git

<https://docs.oracle.com/en/database/oracle/sql-developer-data-modeler/23.1/dmdug/data-modeler-concepts-usage.html>

29

LES OPÉRATIONS DE BASE

LDD

Création d'une table

CREATE TABLE nom_table(élément_de_table1, élément_de_table2, ..., élément_de_tableN)

avec

- élément_de_table = nom_de_colonne type_donnée [clause_défaut] [contrainte_de_colonne]
- **clause_défaut**
 - DEFAULT valeur

32

Contraintes d'intégrité

La contrainte_de_colonne porte sur un seul attribut

- NOT NULL: Valeur nulle impossible
- UNIQUE ou PRIMARY KEY unicité de l'attribut (UNIQUE garantit l'unicité et PRIMARY KEY définit la clé primaire de la relation)
- REFERENCES Contraintes référentielles REFERENCES Table_référencée [(colonne_référencée)]
- CHECK Contraintes générales : CHECK (condition)

33

Contraintes d'intégrité

Contrainte de relation

- UNIQUE / PRIMARY KEY **Contraintes d'unicité**
 - UNIQUE (attribut1, attribut2, ..., attributN)
 - PRIMARY KEY (attribut1, attribut2, ..., attributN)
- REFERENCES **Contraintes référentielles**: spécifient quels attributs référencent ceux d'une autre table
 - [FOREIGN KEY (colonne_référencante1, ..., colonne_référencanteN)]
 - REFERENCES table_référencée [(colonne_référencée1, ..., colonne_référencéeN)]
 - [ON DELETE CASCADE]
- CHECK **Contraintes générales** CHECK (condition)

34

Création de table

CREATE TABLE nomTable AS SELECT * FROM TABLE1

CREATE TABLE nomTable AS SELECT * FROM TABLE1 WHERE Contrainte1 = Contrainte2

- Ex. : **CREATE TABLE** nomTable AS SELECT * FROM TABLE1 WHERE 1 = 2

35

Contraintes d'intégrité

ON DELETE CASCADE

- Ex.1

```
CREATE TABLE emp (  
...  
dept integer  
constraint r_dept references dept(dept))
```
- Ex.2

```
CREATE TABLE emp (  
...  
dept integer  
constraint r_dept references dept(dept)  
on delete cascade)
```

36

Contraintes d'intégrité

ON DELETE SET NULL permet définir une valeur NULL pour les lignes enfant lorsqu'une clé parent est supprimée

Ex.

```
ALTER TABLE emp2 ADD CONSTRAINT emp_dt_fk  
FOREIGN KEY (Department_id)  
REFERENCES departments(department_id) ON DELETE SET NULL;
```

37

Contraintes

Les contraintes sont définies dans la commande

- CREATE (ou ALTER) TABLE
- à l'intérieur des définitions de colonnes pour les contraintes de colonne
 - CONSTRAINT *nomContrainte defContrainte* possible
- au même niveau que définitions de colonnes pour les contraintes de relation
 - CONSTRAINT *nomContrainte defContrainte*

38

C.I. complexes

Assertions générales

- Une assertion est une formule logique qui doit être vraie quelle que soit l'extension de la BD
- Déclencheurs (Triggers) : événement - condition – action.
 - Action déclenchée suite à la réalisation de l'événement, si la condition est vérifiée.
 - Action : vérification ou mise à jour

39

Exemple Oracle

```
CREATE TABLE hr.admin_emp (
    empno    NUMBER(5) PRIMARY KEY,
    ename    VARCHAR2(15) NOT NULL,
    ssn      NUMBER(9) ENCRYPT,
    job      VARCHAR2(10),
    mgr      NUMBER(5),
    hiredate DATE DEFAULT (sysdate),
    photo    BLOB,
    sal      NUMBER(7,2),
    hrly_rate NUMBER(7,2) GENERATED
    ALWAYS AS (sal/2080),
    comm     NUMBER(7,2),
    deptno   NUMBER(3) NOT NULL
    CONSTRAINT admin_dept_fkey
    REFERENCES hr.departments
    (department_id))
    TABLESPACE admin_tbs
```

40

Modification des contraintes

Ex.1

```
◦ ALTER TABLE emp
  DROP CONSTRAINT nom_unique
  ADD (CONSTRAINT ....);
```

Ex.2

```
◦ ALTER TABLE emp
  MODIFY nomEmployé constraint sal_min
  check(...);
```

41

Renommer des colonnes et des contraintes de table

RENAME COLUMN de l'instruction ALTER TABLE permet de renommer les colonnes d'une table donnée

◦ Exemple Oracle :

```
ALTER TABLE marketing RENAME COLUMN team_id
TO id;
```

RENAME CONSTRAINT de l'instruction ALTER TABLE permet de renommer n'importe quelle contrainte existante pour une table donnée

◦ Exemple Oracle :

```
ALTER TABLE marketing RENAME CONSTRAINT mktg_pk
TO new_mktg_pk;
```

42

Contraintes

```
CONSTRAINT cstr definition_contrainte
[NOT] DEFERRABLE
```

Valeur par défaut: NOT DEFERRABLE

```
[ [NOT] DEFERRABLE [INITIALLY {IMMEDIATE | DEFERRED}] ]
| INITIALLY { IMMEDIATE | DEFERRED } [ NOT ] [ DEFERRABLE ] ]
[ RELY | NORELY ] [ using_index_clause ]
[ ENABLE | DISABLE ] [ VALIDATE | NOVALIDATE ]
[ exceptions_clause ]
```

43

Contraintes

Pour différer les contraintes: set CONSTRAINTS { liste de contraintes | ALL } DEFERRED

44

Création d'INDEX

```
CREATE [ UNIQUE | BITMAP ] INDEX [ schema. ] index
ON { cluster_index_clause
    | table_index_clause
    | bitmap_join_index_clause
}
[ USABLE | UNUSABLE ]
[ { DEFERRED | IMMEDIATE } INVALIDATION ] ;
```

45

Exemple Oracle

```
CREATE INDEX ord_customer_ix
ON orders (customer_id);
```

46

Suppression des Tables

```
DROP TABLE nom_de_table [PURGE]
```

47

Modifier la définition d'une table

On peut ajouter une nouvelle colonne ou modifier la définition d'une colonne

```
ALTER TABLE table ADD (col1 type1, col2 type2,...)
```

```
ALTER TABLE table MODIFY (col1 type1, col2 type2,...)
```

On ne peut modifier une colonne que si la colonne ne contient que des valeurs null ou si la nouvelle définition est compatible avec les valeurs déjà entrées dans cette colonne

48

Vue

On peut enregistrer un select en tant que vue

Les utilisateurs pourront consulter ou modifier la base (avec restrictions) à travers la vue comme si c'était une table réelle

```
CREATE VIEW vue [(col1, col2,...)] AS select ... [WITH CHECK OPTION]
```

Le select peut contenir toutes les clauses d'un select sauf "order by"

Ex. create view emp10 as select * from emp where dept = 10;

```
DROP VIEW vue
```

49

Création de Schéma

Syntaxe :

```
CREATE SCHEMA AUTHORIZATION schema
```

```
{ create_table_statement
```

```
| create_view_statement
```

```
| grant_statement
```

```
}...
```

```
;
```

50

Exemple Oracle

```
CREATE SCHEMA AUTHORIZATION oe
```

```
CREATE TABLE new_product
```

```
(color VARCHAR2(10) PRIMARY KEY, quantity NUMBER)
```

```
CREATE VIEW new_product_view
```

```
AS SELECT color, quantity FROM new_product WHERE color = 'RED'
```

```
GRANT select ON new_product_view TO hr;
```

51

Synonyme pour un objet

Simplifie l'accès aux objets. Permet de créer un alias

- Une référence plus simple à une table appartenant à un autre utilisateur
- Un raccourci les noms d'objet trop longs

Syntaxe : `CREATE [PUBLIC] SYNONYM synonym`
 `FOR object;`

Exemple Oracle pour la vue DEPT_SUM_VU :

- `CREATE SYNONYM d_sum`
- `FOR dept_sum_vu;`

52

Suppression d'un synonyme

Syntaxe : `DROP SYNONYM nom_synonyme`

Ex. Oracle : `DROP SYNONYM d_sum;`

53

LMD

Manipulation des données

`INSERT` pour ajouter des n-uplets

`UPDATE` pour modifier des n-uplets

`DELETE` pour supprimer des n-uplets

55

Insertion

INSERT INTO nom_table [(nom_de_colonne1, ..., nom_de_colonneN)]

VALUES (Valeur1, ... ValeurN)

ou

INSERT INTO nom_table [(nom_de_colonne1, ..., nom_de_colonneN)]

SELECT Attribut1, ..., AttributN

WHERE ...

56

Insertion

Liste des colonnes optionnelle

Si omission d'une colonne ==> valeur NULL

57

WITH CHECK OPTION

Empêche le changement des lignes qui ne sont pas dans la sous-requête

Exemple Oracle

```
INSERT INTO ( SELECT location_id, city, country_id
              FROM   loc
              WHERE  country_id IN
                    (SELECT country_id
                     FROM countries
                     NATURAL JOIN regions
                     WHERE region_name = 'Europe')
              WITH CHECK OPTION )
VALUES (3600, 'Washington', 'US');
```

→ VALUES (3500, 'Berlin', 'DE'); fonctionne

58

Avec une Vue

Création d'une vue

```
CREATE OR REPLACE VIEW european_cities
AS
SELECT location_id, city, country_id
FROM   locations
WHERE  country_id in
      (SELECT country_id
       FROM countries
       NATURAL JOIN regions
       WHERE region_name = 'Europe')
WITH CHECK OPTION;
```

59

Exemple

```
INSERT INTO european_cities VALUES (3400,'New York','US');
```

60

Default

le mot clé DEFAULT comme valeur de colonne où la valeur de colonne par défaut est souhaitée.

Les valeurs par défaut explicites peuvent être utilisées dans les instructions INSERT et UPDATE

61

Exemple Oracle

INSERT:

```
INSERT INTO deptm3
(department_id, department_name, manager_id)
VALUES (300, 'Engineering', DEFAULT);
```

UPDATE:

```
UPDATE deptm3
SET manager_id = DEFAULT
WHERE department_id = 10;
```

62

Modification

UPDATE nom_table

SET nom_de_colonne1=expression_de_valeur1, ...,

nom_de_colonneN= expression_de_valeurN

WHERE condition_de_recherche

L'expression_de_valeur peut être positionnée à NULL

63

Suppression

DELETE FROM *nom_table*

[**WHERE** *condition_de_recherche*]

64

MERGE

Permet de faire une insertion ou une mäj conditionnelle

Réalise

- Une mäj si le n-upplet existe
- Une insertion si c'est un nouveau n-upplet

Evite les mäj séparées

Augmente les performances

Utilisé dans les applications de data warehouse

65

Syntaxe

```
MERGE INTO nom_table table_alias
USING (table|view|sub_query) alias
ON (join condition)
WHEN MATCHED THEN
UPDATE SET
  col1 = col1_val1,
  col2 = col2_val
WHEN NOT MATCHED THEN
INSERT (column_list)
VALUES (column_values);
```

66

Exemple

```
MERGE INTO copy_emp c
USING employees e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
  c.first_name = e.first_name,
  c.last_name = e.last_name,
  ...
  c.department_id = e.department_id
```

```
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name,
e.last_name,
e.email, e.phone_number, e.hire_date,
e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

67

Interrogation

Expression d'une requête par un bloc SFW: **SELECT FROM WHERE**

Syntaxe générale

```
SELECT [ALL|DISTINCT] attribut(s) [[AS] ALIAS] FROM table(s)
[WHERE condition]
[GROUP BY attribut(s) [HAVING condition]]
[ORDER BY attribut(s) [ASC|DESC]];
```

68

Conditions

Les conditions de base sont exprimées de deux façons:

- *attribut comparateur valeur*
- *attribut comparateur attribut*
- où comparateur est =, <, <=, <>, >=, BETWEEN, LIKE, IS NULL, IN

Soit le schéma de relation FOURNITURE(PNOM,FNOM,PRIX)

REQUÊTE: *Produits de prix supérieure à 200€*

```
SELECT PNOM
FROM FOURNITURE
WHERE PRIX > 200
```

69

Exemple

Soit le schéma de relation FOURNITURE(PNOM,FNOM,PRIX)

REQUÊTE: *Produits avec un coût entre 1000€ et 2000€*

SQL:

```
SELECT PNOM
FROM FOURNITURE
WHERE PRIX BETWEEN 1000 AND 2000
```

Rmq: $y \text{ BETWEEN } x \text{ AND } z \Leftrightarrow y \geq x \text{ AND } y \leq z$

70

Restriction

WHERE ROWNUM <= n

- Permet de limiter le nombre de résultats à n valeurs mais Attention :
- Exemples :
 - SELECT * FROM HR.EMPLOYEES WHERE ROWNUM <= 5 ORDER BY SALARY DESC;
 - SELECT * FROM (SELECT * FROM HR.EMPLOYEES ORDER BY SALARY DESC) WHERE ROWNUM <= 5;

FETCH [FIRST|NEXT] n ROWS ONLY

- Exemples :
 - SELECT * FROM HR.EMPLOYEES ORDER BY SALARY DESC FETCH FIRST 5 ROWS ONLY;

71

Résultats

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	1.515.555.0100	2013-06-17T00:00:00Z	AD_PRES	24000			90
101	Neena	Yang	NYANG	1.515.555.0101	2015-09-21T00:00:00Z	AD_VP	17000		100	90
102	Lex	Garcia	LGARCIA	1.515.555.0102	2011-01-13T00:00:00Z	AD_VP	17000		100	90
103	Alexander	James	AJAMES	1.590.555.0103	2016-01-03T00:00:00Z	IT_PROG	9000		102	60
104	Bruce	Miller	BMILLER	1.590.555.0104	2017-05-21T00:00:00Z	IT_PROG	6000		103	60
EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	1.515.555.0100	2013-06-17T00:00:00Z	AD_PRES	24000			90
101	Neena	Yang	NYANG	1.515.555.0101	2015-09-21T00:00:00Z	AD_VP	17000		100	90
102	Lex	Garcia	LGARCIA	1.515.555.0102	2011-01-13T00:00:00Z	AD_VP	17000		100	90
145	John	Singh	JSINGH	44.1632.960000	2014-10-01T00:00:00Z	SA_MAN	140000.4		100	80
146	Karen	Partners	KPARTNER	44.1632.960001	2015-01-05T00:00:00Z	SA_MAN	135000.3		100	80
EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	1.515.555.0100	2013-06-17T00:00:00Z	AD_PRES	24000			90
101	Neena	Yang	NYANG	1.515.555.0101	2015-09-21T00:00:00Z	AD_VP	17000		100	90
102	Lex	Garcia	LGARCIA	1.515.555.0102	2011-01-13T00:00:00Z	AD_VP	17000		100	90
145	John	Singh	JSINGH	44.1632.960000	2014-10-01T00:00:00Z	SA_MAN	140000.4		100	80
146	Karen	Partners	KPARTNER	44.1632.960001	2015-01-05T00:00:00Z	SA_MAN	135000.3		100	80

72

Sélection aléatoire

SELECT ... SAMPLE(n);

- Exemple
 - SELECT * FROM HR.EMPLOYEES SAMPLE(5);

73

Like

Soit le schéma de relation COMMANDES(NUM,CNOM,PNOM,QUANTITE)

REQUÊTE: Clients dont le nom commence par "C"

```
SELECT CNOM
FROM COMMANDES
WHERE CNOM LIKE 'C%'
```

Rmq:

- Le littéral qui suit LIKE doit être une chaîne de caractères éventuellement avec des caractères jokers (__,%). Impossible avec l'algèbre relationnel
- Les jokers peuvent être différents selon les SGBD (? et *)

74

Null

Une valeur "spéciale" qui représente une valeur (information) inconnue.

A Op B est inconnu (ni vrai, ni faux) si

- A ou(et) B est NULL (avec Op est =, <, >, >=, <=, ou +, /, *, -)

75

Null

Soit le schéma de relation **FOURNISSEUR(FNOM,STATUT,VILLE)**

REQUÊTE: *Fournisseurs dont l'adresse est inconnu.*

```
SELECT FNOM
FROM FOURNISSEUR
WHERE VILLE IS NULL
```

Rmq: Le prédicat IS NULL (ou IS NOT NULL) n'est pas exprimable en algèbre relationnelle.

76

Alias de colonne

Renomme un en-tête de colonne

Utile dans les calculs

Suit immédiatement le nom de colonne (précédé du mot facultatif AS)

Entre guillemets ("") s'il contient des espaces ou des caractères spéciaux, ou pour distinguer majuscules et minuscules

Pour faire référence à des colonnes de nom identique mais dans des tables différentes

77

Exemples

```
SELECT last_name AS name, commission_pct comm
FROM employees;
```

```
SELECT last_name "Name" , salary*12 "Annual Salary"
FROM employees;
```

78

In

Soit le schéma de relation **FOURNITURE(PNOM,FNOM,PRIX)**

REQUÊTE: *Produits avec un coût de 100€, de 200€ ou de 300€*

```
SELECT PNOM
FROM FOURNITURE
WHERE PRIX IN {100,200,300}
```

79

Jointure

Syntaxe :

```
SELECT table1.Attribut1, table2.AttributP
FROM table1, table2
WHERE table1.AttributK
CONDITION
table2.AttributM;
```

Jointure : Condition AdHoc

Equijointure : =

Jointure Naturelle : = avec attributs identiques

80

Jointure

Soit le schéma de relations

- COMMANDES(NUM,CNOM,PNOM,QUANTITE)
- FOURNITURE(PNOM,FNOM,PRIX)

REQUÊTE: *Nom, Coût, Fournisseur des Produits commandés par Jean*

- ALGÈBRE: $\pi_{PNOM, PRIX, FNOM} (\sigma_{CNOM="JEAN"} (COMMANDES) \bowtie (FOURNITURE))$

- SQL :

```
SELECT COMMANDES.PNOM, PRIX, FNOM
FROM COMMANDES, FOURNITURE
WHERE CNOM = 'JEAN' AND
COMMANDES.PNOM = FOURNITURE.PNOM
```

81

Jointure

Exemples

- Employé(**Matricule**, NomEmployé, Poste, DateEmbauche, MatriculeSupérieur, Salaire, NumeroDept)
- Dept(**NumeroDept**, NomDept, Lieu)

```
SELECT NomEmploye, NomDept
FROM Employe, Dept
WHERE Dept.NumeroDept = Employe.NumeroDept
```

82

Jointure

Jointure quelconque: Clause WHERE avec la condition Ad HOC

83

Jointure

2 relations R1 et R2

$R3 = R1 \bowtie R2$ sous la condition $A_k \text{ op } B_q$

SQL « Standard »:

```
SELECT ...  
FROM R1, R2  
WHERE R1.Ak op R2.Bq
```

- Equijointure:
 - op: =
- Si jointure naturelle:
 - op: =
 - Et $A_k \bowtie B_q$

84

Jointure

Syntaxe générale SQL2

```
SELECT table1.att1, table2.att2  
FROM table1  
[NATURAL JOIN table2] |  
[JOIN table2 USING (attK)] |  
[JOIN table2  
ON (table1.attP = table2.attY)] |  
[LEFT|RIGHT|FULL OUTER JOIN table2  
ON (table1.attM = table2.attN)] |  
[CROSS JOIN table2];
```

85

Jointure

Syntaxe SQL2

```
SELECT ...  
FROM R1 JOIN R2  
ON R1.Ak op R2.Bq
```

Rmq: jointure naturelle possible

Double jointure :

```
SELECT ...  
FROM R1  
JOIN R2  
ON R1.Ak op R2.Bq  
JOIN ...  
ON...
```

86

Jointure naturelle

Syntaxe SQL 2:

```
SELECT ...  
FROM R1  
NATURAL JOIN R2;
```

87

Jointure avec condition

Exemple

Tables EMPLOYEES et JOB_GRADES

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e JOIN job_grades j
ON     e.salary
BETWEEN j.lowest_sal AND j.highest_sal;
```

88

Jointure externe

Ex. Dans l'exemple suivant, un département qui n'a pas d'employé n'apparaîtra pas :

```
SELECT nomEmploye, nomDept
FROM   employe, dept
WHERE  employe.numerodept = dept.numero dept
```

Si on veut qu'il apparaisse, on doit utiliser une jointure externe

89

Jointure externe

Syntaxe SQL 2:

```
SELECT ...
FROM   R1 [NATURAL]
{FULL | LEFT | RIGHT} OUTER JOIN R2
ON ...;
```

Exemple:

```
select nomEmploye, nomDept
from   employe LEFT OUTER JOIN dept
ON     employe.numerodept = dept.numerodept
```

90

Exemples

DEPT NODEPT	NOMDEP	EMP NOEMP	NODEPT	SAL
1	Commande	T	1	10 000
2	Admin	J	2	20 000
4	Usine	K	3	15 000

91

Exemple

EMP NATURAL JOIN DEPT

NOEMP	NODEPT	SAL	NOMDEP
T	1	10 000	Commande
J	2	20 000	Admin

EMP NATURAL FULL OUTER JOIN DEPT

NOEMP	NODEPT	SAL	NOMDEP
T	1	10 000	Commande
J	2	20 000	Admin
K	3	15 000	NULL
NULL	4	NULL	Usine

92

Remarques

Préfixes de tables

- Obligatoires si les noms de colonnes sont ambigus
- Améliorent les performances sauf si les noms de table sont longs → utiliser des alias de table
- Exemple

```
SELECT *  
FROM R1 r, R2 s  
WHERE r.Att1=s.Att2
```

93

Syntaxe USING

Pour une équijointure

Syntaxe

```
SELECT ...  
FROM R1 JOIN R2  
USING (Attribut1)
```

94

Syntaxe USING

Alias de colonne obligatoire si ambiguïté

```
SELECT r.AttributP, ...  
FROM R1 r JOIN R2 s  
USING (Attribut1)
```

95

Syntaxe USING

~~Attention : pas d'alias de colonne pour l'attribut du USING~~

```
SELECT r.AttributK, s.AttributN  
FROM R1 r JOIN R2 s  
USING (Attribut1)  
WHERE s.Attribut1=...
```

```
SELECT r.AttributK, s.AttributN  
FROM R1 r JOIN R2 s  
USING (Attribut1)  
WHERE Attribut1=...
```

96

Restreindre avec

WHERE ou AND

```
SELECT ...  
FROM R1  
JOIN R2  
ON ...  
[WHERE|AND] Condition ;
```

97

Auto-jointure

Joindre une table avec elle-même

Exemple

```
SELECT tabEMP.last_name emp, tabMGR.last_name mgr  
FROM employees tabEMP JOIN employees tabMGR  
ON (tabEMP.manager_id = tabMGR.employee_id);
```

98

Produit Cartésien

Généré par une jointure

- Sans condition
- Où la condition est non valide

Clause CROSS JOIN

```
SELECT table1.att1, table2.att2  
FROM table1  
CROSS JOIN table2;
```

99

Expression ensemblistes

Union:

- Requete1 UNION Requete2
- L'union élimine les doublés. Pour les garder on utilise UNION ALL: le résultat contient chaque n-uplet a+b fois, où a et b est le nb d'occurrences du n-uplet dans la 1ere et la 2ème requête

Différence:

- Req1 MINUS Req2 (ou EXCEPT selon les SGBD)
- La différence élimine les doublés. Pour les garder, on utilise EXCEPT ALL: le résultat contient chq n-uplet a-b fois où a et b est le nb d'occurrences du n-uplet dans la 1ere et la 2eme requête (pas de MINUS ALL !)

100

Expression ensemblistes

Intersection

- Requete1 INTERSECT Requete2
- L'intersection élimine les doublés. Pour les garder on utilise INTERSECT ALL : le résultat contient chaque n-uplet min(a,b) fois, où a et b est le nb d'occurrences du n-uplet dans la première et la deuxième requête (pas sous Oracle !)

101

Remarques

Sur les opérations ensemblistes

- Les expressions de SELECT doivent avoir même nb de colonnes et même type de données
- Nom des colonnes affichées : celles du 1er SELECT
- Tous les doublons sont retirés
- Ordre Ascendant si ORDER BY avec un attribut du 1er SELECT
- Affichage d'une colonne qui n'existe pas:
 - Possible avec table Oracle particulière DUAL : ex. SELECT * FROM DUAL;
 - Possible avec fonctions de conversions de données

DUMMY
X

102

Exemple

```
SELECT deptno, TO_CHAR(null) as location, hiredate
FROM EMP
UNION
SELECT deptno, loc, TO_DATE(null)
FROM DEPT;
```

103

Requêtes imbriquées

Sous-interrogation ramenant 1 ligne, 1 colonne

- **WHERE expression op (SELECT ...)**
- où op est un des opérateurs de comparaison =, !=, <, >, <=, >=
- Exemple :

```
SELECT nomE from emp
WHERE sal >= (select sal FROM emp where nomE = 'Mercier')
```

104

Requêtes imbriquées

Sous-interrogation ramenant plusieurs lignes

- **WHERE expression op ANY (SELECT ...)**
- **WHERE expression op ALL (SELECT ...)**
- **WHERE expression IN (SELECT ...)**
- **WHERE expression NOT IN (SELECT ...)**
- où op est un des opérateurs de comparaison =, !=, <, >, <=, >=
- **ANY** : vrai si la comparaison est vraie pour au moins une des valeurs ramenées par le **SELECT**
- **ALL** : vrai si la comparaison est vraie pour toutes les valeurs

105

Exemple

```
SELECT nomE, sal
FROM emp
WHERE sal > all
      (SELECT sal
       FROM emp
       WHERE dept = 30);
```

106

Requêtes imbriquées

La Jointure s'exprime par deux blocs SFW imbriqués

- **SELECT ... FROM ...**
WHERE Attribut1 IN
(SELECT Attribut1 FROM... WHERE ...)

La Différence s'exprime aussi par deux blocs SFW imbriqués

- **SELECT ... FROM ...**
WHERE Attribut1 NOT IN
(SELECT Attribut1 FROM... WHERE ...)

107

Requêtes imbriquées

Sous-interrogation ramenant 1 ligne de plusieurs colonnes

- **WHERE (expr1, expr2,...) op (SELECT ...)**

- où op est = ou != (mais pas <, >, <=, >=)

- Exemple :

```
SELECT nomE, poste, sal
FROM emp
WHERE (poste, sal) =
      (SELECT poste, sal FROM emp
      WHERE nomE = 'Mercier');
```

Rmq: le select renvoie 1 seule ligne

108

Requêtes imbriquées

Sous-interrogation ramenant plusieurs lignes de plusieurs colonnes

- **WHERE (expr1, expr2,...) op ANY (SELECT ...)**

- **WHERE (expr1, expr2,...) op ALL (SELECT ...)**

- **WHERE (expr1, expr2,...) IN (SELECT ...)**

- **WHERE (expr1, expr2,...) NOT IN (SELECT ...)**

- où op est = ou != (mais pas <, >, <=, >=)

- Exemple :

```
SELECT nomE, poste, sal FROM emp
WHERE (poste, sal) in
      (select poste, sal FROM emp
      WHERE dept = 10);
```

109

Exists

La clause EXISTS est suivie d'une sous-interrogation entre parenthèses, et prend la valeur vrai s'il existe au moins une ligne satisfaisant les conditions de la sous-interrogation

Ex.

```
SELECT nomE FROM emp E
WHERE exists
      (SELECT null FROM emp
      WHERE sup = E.matr);
```

110

Sous-requêtes corrélées

C'est une sous requête évaluée pour chaque ligne traitée par la requête principale

Utilise les valeurs d'une colonne retournée par la requête principale

La corrélation est obtenue en utilisant un élément de la requête externe dans la sous requête

111

Sous-requêtes corrélées

Etapas d'exécution

- Récupération de la ligne à utiliser (requête externe)
- Exécution de la requête interne avec la valeur de la ligne récupérée
- Utilisation de la valeur retournée par la requête interne pour ou non conserver la ligne
- Itération → à il n'existe plus de lignes

112

Sous-requêtes corrélées

Syntaxe

```
SELECT outer1, outer2, ...  
FROM table1 alias1  
WHERE outer1 operateur (SELECT inner1 FROM table2 alias2  
WHERE alias1.exp1=alias2.exp2);
```

113

Division

Soient les relations R(A, B) et S(C) ;

$R \div S = \{ a \in R[A] / \forall b \in S, (a, b) \in R \} = \{ a \in R[A] / \nexists b \in S, (a, b) \notin R \}$

```
SELECT A FROM R R1  
WHERE NOT EXISTS  
  (SELECT C FROM S  
   WHERE NOT EXISTS  
     (SELECT A, B FROM R  
      WHERE A = R1.A AND B = S.C))
```

114

Exemple

Afficher la liste des numéros de département qui ont tous les postes

La solution est R , S où R = EMP [dept, poste] et S = EMP [poste]

```
SELECT DISTINCT DEPT FROM EMP E  
WHERE NOT EXISTS  
  (SELECT POSTE FROM EMP E2  
   WHERE NOT EXISTS  
     (SELECT DEPT, POSTE FROM EMP  
      WHERE DEPT = E.DEPT  
      AND POSTE = E2.POSTE))
```

115

Fonctions de groupe

SELECT ... FROM... WHERE... GROUP BY...

HAVING ...

Les fonctions de groupe peuvent apparaître dans une expression du SELECT ou du HAVING

AVG moyenne, SUM somme, MIN plus petite valeur, MAX plus grande valeur, VARIANCE variance, STDDEV écart type, COUNT(*), nombre de lignes, COUNT(col) nombre de valeurs non nulles dans la colonne, COUNT(DISTINCT col) nombre de valeurs distinctes

116

GROUP BY

Permet de regrouper des lignes qui ont les mêmes valeurs pour des expressions

- **GROUP BY** *expression1, expression2,...*

Une seule ligne affichée par regroupement de lignes

Ex. : SELECT dept, count(*) FROM emp GROUP BY dept;

Dans la liste des expressions du select ne peuvent figurer que des caractéristiques liées aux groupes : des fonctions de groupes, des expressions figurant dans le GROUP BY

117

Exemples

```
SELECT DEPT, POSTE, COUNT(*) FROM EMP
GROUP BY DEPT, POSTE;
```

```
SELECT DEPT, COUNT(COMM) FROM EMP
GROUP BY DEPT;
```

118

HAVING

Cette clause sert à sélectionner les groupes : **HAVING** *prédicat*

Le prédicat ne peut porter que sur des caractéristiques de groupe

Ex.
SELECT DEPTNO, COUNT(*) FROM SCOTT.EMP
WHERE JOB = 'CLERK'
GROUP BY DEPTNO
HAVING COUNT(*) >= 1
ORDER BY DEPTNO;

```
SELECT DEPTNO, COUNT(*) FROM SCOTT.EMP
GROUP BY DEPTNO
HAVING COUNT(*) = MAX(COUNT(*));    → ORA-00935
```

119

Order By

La clause ORDER BY précise l'ordre dans lesquelles les lignes d'un SELECT seront données:

ORDER BY expr1 [DESC], expr2 [DESC], ...

On peut aussi donner le numéro de la colonne qui servira de clé de tri

Exemples :

- SELECT DEPT, NOMD FROM DEPT ORDER BY NOMD;
- SELECT DEPT, NOMD FROM DEPT ORDER BY 2;

120

Clause PARTITION BY

Utilisée avec les fonctions analytiques (SUM, AVG, RANK ...)

- Syntaxe OVER(PARTITION BY ...)
- pour diviser les résultats en groupes logiques, appelés partitions,
- sur lesquels une fonction est appliquée indépendamment des autres groupes

Fonctions

- SUM(), AVG(), COUNT() → cumul par groupe
- RANK et DENSE_RANK → rang
- ROW_NUMBER() → numéro de ligne dans chaque partition
- LEAD() / LAG() → accès à la ligne suivante/précédente dans le groupe
- FIRST_VALUE() / LAST_VALUE() → première/dernière valeur dans le groupe
- ROWS BETWEEN → pour définir une fenêtre mobile autour de chaque ligne, sur laquelle la fonction va s'appliquer syntaxe : ROWS BETWEEN <début> AND <fin>

121

Début et Fin dans ROWS BETWEEN

Valeurs possibles pour <début> et <fin>

Valeur	Détail
UNBOUNDED PRECEDING	Depuis le début de la partition
CURRENT ROW	La ligne actuelle
n PRECEDING	n lignes avant la ligne actuelle
n FOLLOWING	n lignes après la ligne actuelle
UNBOUNDED FOLLOWING	Jusqu'à la fin de la partition

122

Exemple

Classement des employés par département selon le salaire

```
SELECT employee_id, department_id, salary, RANK() OVER (PARTITION BY
department_id ORDER BY salary DESC) AS salary_rank FROM HR.EMPLOYEES;
```

EMPLOYEE_ID	DEPARTMENT_ID	SALARY	SALARY_RANK
200	10	4400	1
201	20	13000	1
202	20	6000	2
114	30	11000	1
115	30	3100	2
116	30	2900	3
117	30	2800	4
118	30	2600	5
119	30	2500	6
...			
108	100	12008	1
109	100	9000	2
110	100	8200	3
112	100	7800	4
111	100	7700	5
113	100	6900	6
205	110	12008	1
206	110	8300	2
178		7000	1

123

Exemple

Cumul des salaires par département (avec affichage de tous les employés)

```
SELECT employee_id, department_id, salary, SUM(salary) OVER
(PARTITION BY department_id) AS total_dept_salary FROM
HR.EMPLOYEES;
```

EMPLOYEE_ID	DEPARTMENT_ID	SALARY	TOTAL_DEPT_SALARY
200	10	4400	4400
201	20	13000	19000
202	20	6000	19000
114	30	11000	24900
115	30	3100	24900
116	30	2900	24900
117	30	2800	24900
118	30	2600	24900
119	30	2500	24900
203	40	6500	6500
...			
100	90	24000	58000
101	90	17000	58000
102	90	17000	58000
108	100	12008	51608
109	100	9000	51608
110	100	8200	51608
111	100	7700	51608
112	100	7800	51608
113	100	6900	51608
205	110	12008	20308
206	110	8300	20308
178		7000	7000

125

Exemple

Accéder à la ligne précédente avec LAG()

```
SELECT employee_id, department_id, salary, LAG(salary) OVER
(PARTITION BY department_id ORDER BY hire_date) AS
previous_salary FROM HR.EMPLOYEES;
```

EMPLOYEE_ID	DEPARTMENT_ID	SALARY	PREVIOUS_SALARY
200	10	4400	
201	20	13000	
202	20	6000	13000
114	30	11000	
115	30	3100	11000
117	30	2800	3100
116	30	2900	2800
118	30	2600	2900
119	30	2500	2600
203	40	6500	
122	50	7900	
137	50	3600	7900
141	50	3500	3600
184	50	4200	3500
192	50	4000	4200
133	50	3300	4000
...			
109	100	9000	
108	100	12008	9000
110	100	8200	12008
111	100	7700	8200
112	100	7800	7700
113	100	6900	7800
205	110	12008	
206	110	8300	12008
178		7000	

126

Exemple

Première valeur dans chaque groupe

```
SELECT employee_id, department_id, salary,
FIRST_VALUE(salary) OVER (PARTITION BY department_id
ORDER BY hire_date) AS first_salary
FROM HR.EMPLOYEES;
```

EMPLOYEE_ID	DEPARTMENT_ID	SALARY	FIRST_SALARY
200	10	4400	4400
201	20	13000	13000
202	20	6000	13000
114	30	11000	11000
115	30	3100	11000
116	30	2900	11000
117	30	2800	11000
118	30	2600	11000
119	30	2500	11000
203	40	6500	6500
120	50	8000	7900
121	50	8200	7900
122	50	7900	7900
123	50	6500	7900
...			
108	100	12008	9000
109	100	9000	9000
110	100	8200	9000
111	100	7700	9000
112	100	7800	9000
113	100	6900	9000
205	110	12008	8300
206	110	8300	8300
178		7000	7000

127

Exemple

Moyenne mobile sur 3 lignes (actuelle + 2 suivantes)

```
SELECT employee_id, department_id, hire_date,
salary, AVG(salary) OVER ( PARTITION BY
department_id ORDER BY hire_date ROWS
BETWEEN CURRENT ROW AND 2 FOLLOWING ) AS
moving_avg_salary FROM HR.EMPLOYEES;
```

EMPLOYEE_ID	DEPARTMENT_ID	HIRE_DATE	SALARY	MOVING_AVG_SALARY
200		102013-09-17T00:00:00Z	4400	
201		202014-02-17T00:00:00Z	13000	9500
202		202015-08-17T00:00:00Z	6000	6000
114		302012-12-07T00:00:00Z	110005633.333333333333	
115		302013-05-18T00:00:00Z	31002933.3333333333335	
117		302015-07-24T00:00:00Z	28002766.6666666666665	
116		302016-11-15T00:00:00Z	29002656.6666666666665	
118		302017-08-10T00:00:00Z	2600	2550
119		302017-08-10T00:00:00Z	2500	2500
...				
109		1002012-08-16T00:00:00Z	9000	9736
108		1002012-08-17T00:00:00Z	120089302.666666666666	
110		1002015-09-28T00:00:00Z	8200	7900
111		1002015-09-30T00:00:00Z	77007466.666666666667	
112		1002016-03-07T00:00:00Z	7800	7350
113		1002017-12-07T00:00:00Z	6900	6900
205		1102012-06-07T00:00:00Z	12008	10154
206		1102012-06-07T00:00:00Z	8300	8300
178		2017-05-24T00:00:00Z	7000	7000

128

Exemples d'intervalles

Intervalle	Description
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Moyenne cumulative jusqu'à la ligne actuelle
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW	Moyenne glissante sur 3 lignes (2 avant + actuelle)
ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING	Moyenne glissante sur la ligne actuelle + 2 suivantes
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	Moyenne centrée sur 3 lignes (avant, actuelle, après)
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	Moyenne sur toute la partition

129

Fonction GROUP BY avec les opérateurs ROLLUP et CUBE

Utilisez l'opérateur ROLLUP ou CUBE avec la fonction GROUP BY afin de générer des lignes de superagrégation en effectuant des références croisées de colonnes.

Un regroupement avec l'opérateur ROLLUP génère un ensemble de résultats contenant les lignes regroupées et les sous-totaux.

Un regroupement avec l'opérateur CUBE génère un ensemble de résultats contenant les lignes générées par l'opérateur ROLLUP ainsi que les lignes intercroisées.

132

Fonctions avancées

ROLLUP et CUBE

- Permettent d'obtenir des SUPER agrégats de lignes par références croisées aux colonnes

- **Syntaxe:**

```
SELECT  
FROM Table  
[WHERE conditions]  
[GROUP BY [ROLLUP|CUBE] (expn_du_group_by)];
```

133

ROLLUP

Cet opérateur crée des groupements en parcourant dans une direction, « de la droite vers la gauche », la liste des colonnes spécifiées dans le GROUP BY puis effectue la fonction de groupe à ces groupements

Crée des sous-totaux

134

Exemple Oracle

```
SELECT department_id, job_id, SUM(salary)
FROM hr.employees
WHERE department_id < 60
GROUP BY ROLLUP(department_id, job_id);
```

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
20	MK_MAN	13000
20	MK_REP	6000
30	PU_CLERK	13900
30	PU_MAN	11000
40	HR_REP	6500
50	SH_CLERK	64300
50	ST_CLERK	55700
50	ST_MAN	36400

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
10	-	4400
20	MK_MAN	13000
20	MK_REP	6000
20	-	19000
30	PU_MAN	11000
30	PU_CLERK	13900
30	-	24900
40	HR_REP	6500
40	-	6500
50	ST_MAN	36400
50	SH_CLERK	64300
50	ST_CLERK	55700
50	-	156400
-	-	211200

135

CUBE

Retourne toutes les combinaisons possibles des groupes spécifiés ds le GROUP BY

Toutes les valeurs retournées sont calculées à partir de la fonction de groupe spécifiée dans la liste du SELECT

CUBE retourne le même résultat que ROLLUP + la fonction de groupe appliquée au sous-groupe

136

Exemple Oracle

```
SELECT department_id, job_id, SUM(salary)
FROM hr.employees
WHERE department_id < 60
GROUP BY CUBE(department_id, job_id);
```

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
20	MK_MAN	13000
20	MK_REP	6000
30	PU_CLERK	13900
30	PU_MAN	11000
40	HR_REP	6500
50	SH_CLERK	64300
50	ST_CLERK	55700
50	ST_MAN	36400

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
10	-	4400
20	MK_MAN	13000
20	MK_REP	6000
20	-	19000
30	PU_MAN	11000
30	PU_CLERK	13900
30	-	24900
40	HR_REP	6500
40	-	6500
50	ST_MAN	36400
50	SH_CLERK	64300
50	ST_CLERK	55700
50	-	156400
-	-	211200

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
-	-	211200
-	HR_REP	6500
-	MK_MAN	13000
-	MK_REP	6000
-	PU_MAN	11000
-	ST_MAN	36400
-	AD_ASST	4400
-	PU_CLERK	13900
-	SH_CLERK	64300
-	ST_CLERK	55700
10	-	4400
10	AD_ASST	4400
20	-	19000
20	MK_MAN	13000
20	MK_REP	6000
20	-	19000
30	-	24900
30	PU_MAN	11000
30	PU_CLERK	13900
40	-	6500
40	HR_REP	6500
50	-	156400
50	ST_MAN	36400
50	SH_CLERK	64300
50	ST_CLERK	55700

137

Fonction GROUPING

Avec CUBE et ROLLUP: apparition de champs vides

Pour éviter de les confondre avec des champs vides: fonction GROUPING

```
SELECT att1, fonction_grpe, GROUPING (expn_a_1_argument)
FROM Table
[WHERE condition]
[GROUP BY [ROLLUP-CUBE] (expn_grpe) ]
```

Permet de déterminer les groupes impliqués dans le sous-total d'une ligne

138

Fonction GROUPING

Se comporte comme une fonction booléenne

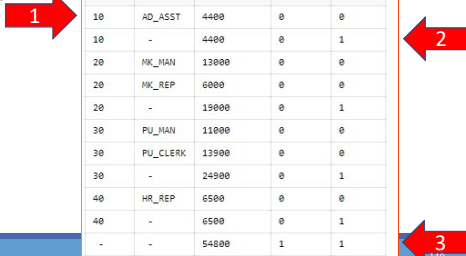
Retour:

- 0: le champ a été utilisé pour calculer le résultats de la fonction, la valeur NULL dans le champs correspond à une valeur NULL dans la Table
- 1: le champ n'a pas été utilisé pour calculer le résultat de la fonction, la valeur NULL dans le champ a été créée par ROLLUP/CUBE à la suite d'un groupement

139

Exemple Oracle

```
SELECT department_id DEPTID,  
       job_id JOB, SUM(salary),  
       GROUPING(department_id)  
       GRP_DEPT,  
       GROUPING(job_id) GRP_JOB  
FROM   hr.employees  
WHERE  department_id < 50  
GROUP BY ROLLUP(department_id,  
                job_id);
```



DEPTID	JOB	SUM(SALARY)	GRP_DEPT	GRP_JOB
10	AD_ASST	4400	0	0
10	-	4400	0	1
20	MK_MAN	13000	0	0
20	MK_REP	6000	0	0
20	-	19000	0	1
30	PU_MAN	11000	0	0
30	PU_CLERK	13900	0	0
30	-	24900	0	1
40	HR_REP	6500	0	0
40	-	6500	0	1
-	-	54800	1	1

140

GROUPING SETS

Permet de définir plusieurs regroupements dans la même interrogation

- Tous sont calculés
- Les résultats des regroupements individuels sont combinés avec une opération UNION ALL

Un seul passage sur la table de base

Instructions UNION complexes inutiles

Plus la clause GROUPING SETS possède d'éléments, plus le gain en performances est élevé

141

Exemple Oracle

```
SELECT department_id, job_id,  
       manager_id,AVG(salary)  
FROM   hr.employees  
GROUP BY GROUPING SETS  
((department_id,job_id),  
(job_id,manager_id));
```



	DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVG(SALARY)
1	(null)	SH_CLERK	122	3200
2	(null)	AC_MGR	101	12000
3	(null)	ST_MAN	100	7280
4	(null)	ST_CLERK	121	2675

	DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVG(SALARY)
39	110	AC_MGR	(null)	12000
40	90	AD_PRES	(null)	24000
41	60	IT_PROG	(null)	5760
42	100	FL_MGR	(null)	12000

142

Exemple équivalent

```
SELECT department_id, job_id, NULL as manager_id,
       AVG(salary) as AVGSAL
FROM hr.employees
GROUP BY department_id, job_id
UNION ALL
SELECT NULL, job_id, manager_id, avg(salary) as AVGSAL
FROM employees
GROUP BY job_id, manager_id;
```

143

Exemple Oracle

```
SELECT department_id, job_id,
       manager_id, AVG(salary)
FROM employees
GROUP BY
GROUPING SETS
((department_id, job_id,
  manager_id),
 (department_id, manager_id),
 (job_id, manager_id));
```

DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVGSAL
90	AD_VP	-	17000
100	FT_MGR	-	12000
80	SA_REP	-	8396.55172413793103448275062060965172414
-	SA_REP	-	7000
90	AD_PRES	-	24000
20	HR_REP	-	6000
110	AC_MGR	-	12000
60	IT_PROG	-	5760
30	PU_CLERK	-	2780
80	SA_MAN	-	12200
50	SH_CLERK	-	3215
20	HR_MAN	-	12000
30	PU_MAN	-	11000
50	ST_CLERK	-	2785
70	PR_REP	-	10000
110	AC_ACCOUNT	-	8300
50	ST_MAN	-	7280
100	FT_ACCOUNT	-	7920
10	AD_ASST	-	4400
40	HR_REP	-	6500
-	ST_CLERK	124	2925
-	SA_MAN	100	12200
-	AC_MGR	101	12000
-	FT_ACCOUNT	108	7920
-	HR_REP	201	6000

Exemples équivalents Oracle

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM hr.employees
GROUP BY CUBE(department_id, job_id, manager_id);
```

8 regroupements
(2 * 2 * 2)

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM hr.employees
GROUP BY department_id, job_id, manager_id
UNION ALL
SELECT department_id, NULL, manager_id, AVG(salary)
FROM hr.employees
GROUP BY department_id, manager_id
UNION ALL
SELECT NULL, job_id, manager_id, AVG(salary)
FROM hr.employees
GROUP BY job_id, manager_id;
```

3 « scan » de la table

145

Colonne composite

Une colonne composite est une collection de colonnes traitée comme une unité :

ROLLUP (a, (b,c) , d)

Avec des () dans la clause GROUP BY, on regroupe les colonnes, qui sont traitées comme une unité lors du calcul des opérations ROLLUP ou CUBE.

Avec ROLLUP ou CUBE, les colonnes composites nécessiteraient d'ignorer l'agrégation sur certains niveaux

146

Exemple

GROUP BY ROLLUP(a, b, c) est équivalent à

GROUP BY a, b, c UNION ALL
GROUP BY a, b UNION ALL
GROUP BY a UNION ALL
GROUP BY ()

GROUP BY CUBE((a, b), c) est équivalent à :

GROUP BY a, b, c UNION ALL
GROUP BY a, b UNION ALL
GROUP BY c UNION ALL
GROUP BY ()

147

Exemple

```
SELECT department_id, job_id,
       manager_id,
       SUM(salary)
FROM   hr.employees
GROUP BY ROLLUP(
  department_id, (job_id,
  manager_id));
```

DEPARTMENT_ID	JOB_ID	MANAGER_ID	SUM(SALARY)
-	SA_REP	149	7000
-	-	-	7000
10	AD_ASST	101	4400
10	-	-	4400
20	HR_MGR	100	13000
20	HR_REP	201	6000
20	-	-	19000
30	PUR_MGR	100	12000
30	PUR_CLERK	114	12500
30	-	-	24500
40	HR_REP	101	6500
40	-	-	6500
50	ST_MGR	100	36400
50	SH_CLERK	120	11600
50	SH_CLERK	121	14700
50	SH_CLERK	122	12000
50	SH_CLERK	123	13900
50	SH_CLERK	124	11300
50	ST_CLERK	120	10500
50	ST_CLERK	121	10700
50	ST_CLERK	122	10800
50	ST_CLERK	123	12000
50	ST_CLERK	124	11700
50	-	-	156400
60	IT_PROG	102	9000

148

Exemple équivalent

```
SELECT department_id, job_id, manager_id, SUM(salary)
FROM   hr.employees
GROUP   BY department_id, job_id, manager_id
UNION
SELECT  department_id, TO_CHAR(NULL), TO_NUMBER(NULL), SUM(salary)
FROM    hr.employees
GROUP BY department_id
UNION ALL
SELECT  TO_NUMBER(NULL), TO_CHAR(NULL), TO_NUMBER(NULL), SUM(salary)
FROM    hr.employees
GROUP BY ();
```

149

Regroupement concaténé

Il est possible d'utiliser un ROLLUP et un CUBE dans un GROUP BY

Exemples Oracle

```
SELECT department_id, job_id, manager_id,
       SUM(salary)
FROM   employees
GROUP BY department_id,
       ROLLUP(job_id,
       CUBE(manager_id));
```

150

Exemple Oracle

```
SELECT department_id, job_id, manager_id, SUM(salary)
totsal
FROM employees
WHERE department_id < 60
GROUP BY GROUPING SETS (department_id),
GROUPING SETS (job_id, manager_id);
```

151

Exemples

Soit le schéma de relation **FOURNISSEUR(FNOM,STATUT,VILLE)**

EMPLOYE(EMPNO,ENOM,DEPNO,SAL)

REQUÊTE1 : Nom et Salaire des Employés gagnant plus que l'employé de numéro 12546 en SQL

REQUÊTE2 : Fournisseurs qui habitent deux à deux dans la même ville (sans les doublés du type x,x et sans mettre x,y et y,x) en SQL

152

La clause WITH

Permet d'utiliser le même bloc d'interrogation dans une instruction SELECT lorsqu'il apparaît plusieurs fois dans une interrogation complexe

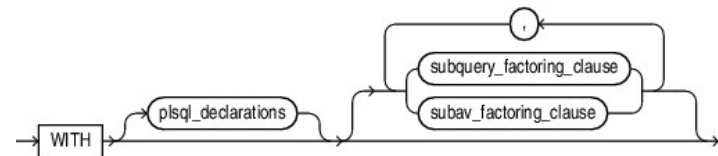
Extrait les résultats d'un bloc d'interrogation et les stocke dans le tablespace temporaire de l'utilisateur

153

La clause WITH

Syntaxe :

- Avec procédures & fonctions PL/SQL → `plsql_declarations`
- Blocs de sous-requêtes
- `subquery_factoring_clause` : attribution d'un nom à une sous-requête, nom utilisable ultérieurement
- `subav_factoring_clause` : vue analytique transitoire



154

Exemple Oracle Clause WITH et PL/SQL

```
WITH
FUNCTION get_domain(url VARCHAR2) RETURN VARCHAR2 IS
    pos BINARY_INTEGER;
    len BINARY_INTEGER;
BEGIN
    pos := INSTR(url, 'www.');
    len := INSTR(SUBSTR(url, pos + 4), '.') - 1;
    RETURN SUBSTR(url, pos + 4, len);
END;
SELECT DISTINCT get_domain(catalog_url)
FROM product_information;
```

155

Exemple

Soit la requête

“Afficher le nom et le total des salaires des départements dont le salaire total est supérieur au salaire moyen de l'ensemble des départements”

Requêtes intermédiaires

- Le salaire total pour chaque département
- Le salaire moyen de l'ensemble des départements
- Comparer le salaire total calculé à la première étape au salaire moyen calculé à la deuxième étape : Si le salaire total d'un département donné est supérieur au salaire moyen de l'ensemble des départements, afficher le nom et le salaire total de ce département.

156

Clause WITH : Exemple Oracle

```
WITH
dept_costs AS (
    SELECT d.department_name, SUM(e.salary) AS dept_total
    FROM   employees e JOIN departments d
    ON     e.department_id = d.department_id
    GROUP BY d.department_name),
avg_cost AS (
    SELECT SUM(dept_total)/COUNT(*) AS dept_avg
    FROM   dept_costs)
SELECT *
FROM   dept_costs
WHERE  dept_total >
      (SELECT dept_avg
       FROM avg_cost)
ORDER BY department_name;
```

DEPARTMENT_NAME	DEPT_TOTAL
Sales	304500
Shipping	156400

157

Clause WITH récursive

Permet la formulation des interrogations récursives

Crée l'interrogation avec un nom, appelé nom d'élément WITH récursif

Contient deux types de membre de blocs d'interrogation : membre d'ancrage et membre récursif

Est conforme à la norme ANSI

158

Exemple Oracle

```
WITH Reachable_From (Source, Destin,
TotalFlightTime) AS (
  SELECT Source, Destin, Flight_time
  FROM Flights
  UNION ALL
  SELECT incoming.Source, outgoing.Destin,
incoming.TotalFlightTime+outgoing.Flight_t
ime
  FROM Reachable_From incoming, Flights
  outgoing
  WHERE incoming.Destin = outgoing.Source
)
SELECT Source, Destin, TotalFlightTime
FROM Reachable_From;
```

	1	2	3	4
	SOURCE	DESTIN	FLIGHT_TIME	
1	San Jose	Los Angeles		1.3
2	New York	Boston		1.1
3	Los Angeles	New York		5.8

	1	2	3	4	5	6
	SOURCE	DESTIN	TOTALFLIGHTTIME			
1	San Jose	Los Angeles		1.3		
2	New York	Boston		1.1		
3	Los Angeles	New York		5.8		
4	San Jose	New York		7.1		
5	Los Angeles	Boston		6.9		
6	San Jose	Boston		8.2		

159

Séquence

Permet de définir des valeurs par défaut

Partageable

Permet de créer des clé primaires sans code additionnel

160

Séquence

Syntaxe:

```
CREATE SEQUENCE nom_seq
[START WITH valeur_debut]
[INCREMENT BY valeur_increment]
[MAXVALUE valeur_maxi | NOMAXVALUE]
[MINVALUE valeur_mini | NOMINVALUE]
[CYCLE | NOCYCLE] [CACHE|NOCACHE]
```

Utilisation:

- nom_seq.nextval
- nom_seq.currval

Modification: ALTER

Suppression: DROP

161

Séquence

Exemple

```
SELECT sequence_name, min_value, max_value,
increment_by, last_number
FROM user_sequences;
```

Utilisation

```
INSERT INTO departments(department_id,
department_name, location_id)
VALUES (dept_deptid_seq.NEXTVAL,
'Support', 2500);
```

162

Séquence

CACHE:

- Accès plus rapide aux valeurs

Trous dans les séquences

- Si rollback
- Crash system
- Si utilisation dans une autre table

NOCACHE:

- Accès à la valeur suivante avec la table USER_SEQUENCES

163

Table temporaire

Permet de stocker une information particulière

```
SELECT ...  
INTO [TEMPORARY | TEMP] [TABLE]  
Nom_nv_table  
FROM ...
```

164

Extensions

INSERT Multitable

Syntaxe:

```
INSERT [ALL] [condition_insert] [clause_insert_into valeurs]  
  (sous_requête)  
condition_insert  
[ALL] [FIRST]  
[WHEN condition THEN] [clause_insert_into valeurs]  
[ELSE] [insert_into_clause valeurs]
```

165

Exemple 1

Sélectionner EMPLOYEE_ID, HIRE_DATE, SALARY, MANAGER_ID dans EMPLOYEES sous la condition EMPLOYEE_ID > à 200.

Insérer ces valeurs dans les tables SAL_HISTORY et MGR_HISTORY

```
INSERT ALL  
  INTO sal_history VALUES (EMPID, HIREDATE, SAL)  
  INTO mgr_history VALUES (EMPID, MGR, SAL)  
SELECT employee_id EMPID, hire_date HIREDATE,  
       salary SAL, manager_id MGR  
FROM employees  
WHERE employee_id > 200;
```

166

Exemple 2

Sélectionner EMPLOYEE_ID, HIRE_DATE, SALARY, MANAGER_ID dans EMPLOYEES sous la condition EMPLOYEE_ID > à 200.

- Si le salaire est > à 10 000 → vers la table SAL_HISTORY
- Si MANAGER_ID > 200 → vers la table MGR_HISTORY

```
INSERT ALL
  WHEN SAL > 10000 THEN
    INTO sal_history VALUES(EMPID,HIREDATE,SAL)
  WHEN MGR > 200 THEN
    INTO mgr_history VALUES(EMPID,MGR,SAL)
SELECT employee_id EMPID,hire_date HIREDATE,
       salary SAL, manager_id MGR
FROM employees
WHERE employee_id > 200;
```

167

Exemple 3

Donner DEPARTMENT_ID , SUM(SALARY) & MAX(HIRE_DATE) dans la table EMPLOYEES

- Si SUM(SALARY) > 25 000 alors insérer dans une table SPECIAL_SAL, en utilisant un INSERT multitable conditionnel
- Si la première clause WHEN est Vrai, la clause WHEN suivante pour ce tuple ne doit pas être évaluée
- Pour les autres tuples qui ne satisfont pas le 1er WHEN insérer dans 1 table HIREDATE_HISTORY_00 ou HIREDATE_HISTORY_99 ou HIREDATE_HISTORY basée sur la valeur de l'attribut HIRE_DATE en utilisant 1 INSERT multitable conditionnel

168

Exemple 3 (2)

```
INSERT FIRST
  WHEN SAL > 25000 THEN
    INTO special_sal VALUES(DEPTID, SAL)
  WHEN HIREDATE like ('%00%') THEN
    INTO hiredate_history_00 VALUES(DEPTID,HIREDATE)
  WHEN HIREDATE like ('%99%') THEN
    INTO hiredate_history_99 VALUES(DEPTID,HIREDATE)
ELSE
  INTO hiredate_history VALUES(DEPTID, HIREDATE)
SELECT department_id DEPTID, SUM(salary) SAL, MAX(hire_date) HIREDATE
FROM employees GROUP BY department_id;
```

169

Dictionnaire des données et Types Oracle

Dictionnaire des données

Ensemble des tables, des objets créés et maintenus par le serveur Oracle

Contient l'information de la BD

Vues disponibles

- USER_*
- ALL_*
- DBA_*

Exemples

```
SELECT *  
FROM dictionary;
```

```
SELECT table_name  
FROM user_tables ;
```

```
SELECT *  
FROM user_catalog ;
```

171

Vues disponibles

Vues DBA_

- Accessible aux personnes disposant du privilège DBA
- Permet de tout visualiser

Vues ALL_

- Accessible à tout le monde
- Permet de visualiser tous les éléments auxquels l'utilisateur est autorisé à accéder.

Vues USER_

- Accessible à tout le monde
- Permet de visualiser tous les éléments dont l'utilisateur est propriétaire

172

Exemple : Vue USER_OBJECTS

```
SELECT object_name, object_type, created, status
```

```
FROM USER_OBJECTS
```

```
ORDER BY object_type;
```

173

Informations sur les tables

USER_TABLES

Name	Null	Type
TABLE_NAME	NOT NULL	VARCHAR2(30)
TABLESPACE_NAME		VARCHAR2(30)
CLUSTER_NAME		VARCHAR2(30)
IOT_NAME		VARCHAR2(30)

```
SELECT table_name FROM USER_TABLES;
```

174

Informations sur les colonnes

USER_TAB_COLUMNS :

Name	Null	Type
TABLE_NAME	NOT NULL	VARCHAR2(30)
COLUMN_NAME	NOT NULL	VARCHAR2(30)
DATA_TYPE		VARCHAR2(106)
DATA_TYPE_MOD		VARCHAR2(3)
DATA_TYPE_OWNER		VARCHAR2(30)
DATA_LENGTH	NOT NULL	NUMBER
DATA_PRECISION		NUMBER
DATA_SCALE		NUMBER
NULLABLE		VARCHAR2(1)

175

Informations sur les colonnes

SELECT column_name, data_type, data_length,
data_precision, data_scale, nullable

FROM USER_TAB_COLUMNS WHERE table_name = 'EMPLOYEES';

	COLUMN_NAME	DATA_TYPE	DATA_LENGTH	DATA_PRECISION
1	EMPLOYEE_ID	NUMBER	22	6
2	FIRST_NAME	VARCHAR2	20	(null)
3	LAST_NAME	VARCHAR2	25	(null)
4	EMAIL	VARCHAR2	25	(null)
5	PHONE_NUMBER	VARCHAR2	20	(null)
6	HIRE_DATE	DATE	7	(null)
7	JOB_ID	VARCHAR2	10	(null)
8	SALARY	NUMBER	22	8
9	COMMISSION_PCT	NUMBER	22	2
10	MANAGER_ID	NUMBER	22	6
11	DEPARTMENT_ID	NUMBER	22	4

176

Informations sur les contraintes

USER_CONSTRAINTS décrit les définitions de contrainte appliquées à vos tables.

USER_CONS_COLUMNS décrit les colonnes dont vous êtes propriétaire et sur lesquelles portent des contraintes.

Name	Null	Type
OWNER	NOT NULL	VARCHAR2(30)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(30)
CONSTRAINT_TYPE		VARCHAR2(1)
TABLE_NAME	NOT NULL	VARCHAR2(30)
SEARCH_CONDITION		LONG()
R_OWNER		VARCHAR2(30)
R_CONSTRAINT_NAME		VARCHAR2(30)
DELETE_RULE		VARCHAR2(9)
STATUS		VARCHAR2(8)
...		

177

Vue USER_CONSTRAINTS : Exemple

SELECT constraint_name, constraint_type, search_condition, r_constraint_name,
delete_rule, status

FROM USER_CONSTRAINTS WHERE table_name = 'EMPLOYEES';

	CONSTRAINT_NAME	C...	SEARCH_CONDITION	R_CONSTR...	DELET...	STATUS
1	EMP_LAST_NAME_NN	C	"LAST_NAME" IS NOT NULL	(null)	(null)	ENABLED
2	EMP_EMAIL_NN	C	"EMAIL" IS NOT NULL	(null)	(null)	ENABLED
3	EMP_HIRE_DATE_NN	C	"HIRE_DATE" IS NOT NULL	(null)	(null)	ENABLED
4	EMP_JOB_NN	C	"JOB_ID" IS NOT NULL	(null)	(null)	ENABLED
5	EMP_SALARY_MIN	C	salary > 0	(null)	(null)	ENABLED
6	EMP_EMAIL_UK	U	(null)	(null)	(null)	ENABLED
7	EMP_EMP_ID_PK	P	(null)	(null)	(null)	ENABLED
8	EMP_DEPT_FK	R	(null)	DEPT_ID_PK	NO ACTION	ENABLED
9	EMP_JOB_FK	R	(null)	JOB_ID_PK	NO ACTION	ENABLED
10	EMP_MANAGER_FK	R	(null)	EMP_EMP_ID_PK	NO ACTION	ENABLED

178

Vue USER_CONS_COLUMNS

```
SELECT constraint_name, column_name
FROM   USER_CONS_COLUMNS
WHERE  table_name = 'EMPLOYEES';
```

	CONSTRAINT_NAME	COLUMN_NAME
1	EMP_LAST_NAME_NN	LAST_NAME
2	EMP_EMAIL_NN	EMAIL
3	EMP_HIRE_DATE_NN	HIRE_DATE
4	EMP_JOB_NN	JOB_ID
5	EMP_SALARY_MIN	SALARY
6	EMP_EMAIL_UK	EMAIL

Name	Null	Type

OWNER	NOT NULL	VARCHAR2(30)
CONSTRAINT_NAME	NOT NULL	VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
COLUMN_NAME		VARCHAR2(4000)
POSITION		NUMBER

179

Informations sur les vues

USER_VIEWS

Name	Null	Type

VIEW_NAME	NOT NULL	VARCHAR2(30)
TEXT_LENGTH		NUMBER
TEXT		LONG()

180

Informations sur les séquences

Vue USER_SEQUENCES

Name	Null	Type

SEQUENCE_NAME	NOT NULL	VARCHAR2(30)
MIN_VALUE		NUMBER
MAX_VALUE		NUMBER
INCREMENT_BY	NOT NULL	NUMBER
CYCLE_FLAG		VARCHAR2(1)
ORDER_FLAG		VARCHAR2(1)
CACHE_SIZE	NOT NULL	NUMBER
LAST_NUMBER	NOT NULL	NUMBER

181

Vérifier les séquences

```
SELECT sequence_name, min_value, max_value,
       increment_by, last_number
```

```
FROM   USER_SEQUENCES;
```

La colonne LAST_NUMBER affiche le prochain numéro de séquence disponible si l'option NOCACHE est indiquée

182

Informations sur les index

Vue USER_INDEXES

Vue USER_IND_COLUMNS décrit les colonnes composant les index et les colonnes d'index des tables de l'utilisateur

Name	Null	Type

INDEX_NAME	NOT NULL	VARCHAR2(30)
INDEX_TYPE		VARCHAR2(27)
TABLE_OWNER	NOT NULL	VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
TABLE_TYPE		VARCHAR2(11)
UNIQUENESS		VARCHAR2(9)

183

Vue USER_INDEXES : Exemples

```
SELECT index_name, table_name, uniqueness
FROM USER_INDEXES
WHERE table_name = 'EMPLOYEES';
```

```
SELECT index_name, table_name
FROM USER_INDEXES
WHERE table_name = 'emp_lib';
```

184

Vue USER_IND_COLUMNS

USER_IND_COLUMNS

Name	Null	Type

INDEX_NAME		VARCHAR2(30)
TABLE_NAME		VARCHAR2(30)
COLUMN_NAME		VARCHAR2(4000)
COLUMN_POSITION		NUMBER
COLUMN_LENGTH		NUMBER
CHAR_LENGTH		NUMBER
DESCEND		VARCHAR2(4)

```
SELECT INDEX_NAME, COLUMN_NAME, TABLE_NAME
FROM USER_IND_COLUMNS
WHERE INDEX_NAME = 'lname_idx';
```

185

Informations sur les synonymes

USER_SYNONYMS

Name	Null	Type

SYNONYM_NAME	NOT NULL	VARCHAR2(30)
TABLE_OWNER		VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
DB_LINK		VARCHAR2(128)

```
SELECT *
FROM USER_SYNONYMS;
```

186

Ajouter des commentaires

Syntaxe : `COMMENT ON {TABLE nom_table | COLUMN nom_colonne} IS '...'`

Ex. Oracle

```
COMMENT ON TABLE employees IS 'Employee Information';
```

```
COMMENT ON COLUMN employees.first_name IS 'First name of  
the employee';
```

Les commentaires sont visibles grâce aux vues

- `ALL_COL_COMMENTS`
- `USER_COL_COMMENTS`
- `ALL_TAB_COMMENTS`
- `USER_TAB_COMMENTS`

187

Type Oracle

Data Type	Description
VARCHAR2 (<i>size</i>)	Variable-length character data
CHAR (<i>size</i>)	Fixed-length character data
NUMBER (<i>p, s</i>)	Variable-length numeric data
DATE	Date and time values
LONG	Variable-length character data up to 2 gigabytes
CLOB	Character data up to 4 gigabytes
RAW and LONG RAW	Raw binary data
BLOB	Binary data up to 4 gigabytes
BFILE	Binary data stored in an external file; up to 4 gigabytes
ROWID	A 64 base number system representing the unique address of a row in its table

188

Type DATE

Data Type	Description
TIMESTAMP	Date en secondes
INTERVAL YEAR TO MONTH	Intervalle en année et mois
INTERVAL DAY TO SECOND	Intervalle de jours en heures minutes et secondes

189

Type DATE

TIMESTAMP WITH TIME ZONE : inclut une zone temporelle

- Time Zone : différence, en heures & minutes, entre temps local & UTC

TIMESTAMP WITH LOCAL TIME ZONE

190

CURRENT_DATE, CURRENT_TIMESTAMP, LOCALTIMESTAMP

CURRENT_DATE :

- Renvoie la date actuelle de la session utilisateur
- Type de données DATE

CURRENT_TIMESTAMP :

- Renvoie la date et l'heure actuelles de la session utilisateur
- Type de données TIMESTAMP WITH TIME ZONE

LOCALTIMESTAMP :

- Renvoie la date et l'heure actuelles de la session utilisateur
- Type de données TIMESTAMP

191

Exemples

```
ALTER SESSION
SET NLS_DATE_FORMAT = 'DD-MON-YYYY HH24:MI:SS';
ALTER SESSION SET TIME_ZONE = '-5:00';

SELECT SESSIONTIMEZONE, CURRENT_DATE FROM DUAL;
SELECT SESSIONTIMEZONE, CURRENT_TIMESTAMP FROM DUAL;
SELECT SESSIONTIMEZONE, LOCALTIMESTAMP FROM DUAL;
```

192

Résultats

SESSIONTIMEZONE	CURRENT_DATE
1 -05:00	23-JUN-2009 01:34:52

SESSIONTIMEZONE	CURRENT_TIMESTAMP
1 -05:00	23-JUN-09 01.35.26.239882000 AM -05:00

SESSIONTIMEZONE	LOCALTIMESTAMP
1 -05:00	23-JUN-09 01.36.21.811798000 AM

193

DBTIMEZONE et SESSIONTIMEZONE

Affichez la valeur du fuseau horaire de la base de données :

```
SELECT DBTIMEZONE FROM DUAL;
```

DBTIMEZONE
1 +00:00

Affichez la valeur du fuseau horaire de la session :

```
SELECT SESSIONTIMEZONE FROM DUAL;
```

SESSIONTIMEZONE
1 -05:00

194

Types de données TIMESTAMP

Type de données	Champs
TIMESTAMP	Year, Month, Day, Hour, Minute, Second with fractional seconds
TIMESTAMP WITH TIME ZONE	Identique au type de données TIMESTAMP ; inclut aussi : TIMEZONE_HOUR et TIMEZONE_MINUTE ou TIMEZONE_REGION
TIMESTAMP WITH LOCAL TIME ZONE	Identique au type de données TIMESTAMP ; sa valeur inclut un décalage horaire

195

Champs TIMESTAMP

Champ date-heure	Valeurs valides
YEAR	-4 712 à 9 999 (à l'exclusion de l'année 0)
MONTH	01 à 12
DAY	01 à 31
HOUR	00 à 23
MINUTE	00 à 59
SECOND	00 à 59.9(N), où 9(N) est la précision
TIMEZONE_HOUR	-12 à 14
TIMEZONE_MINUTE	00 à 59

196

Différence entre DATE et TIMESTAMP

```
-- when hire_date is of type
DATE
```

```
SELECT hire_date
FROM employees;
```

	HIRE_DATE
1	21-JUN-99
2	13-JAN-00
3	17-SEP-87
4	17-FEB-96
5	17-AUG-97
6	07-JUN-94
7	07-JUN-94
8	07-JUN-94

```
ALTER TABLE employees
MODIFY hire_date TIMESTAMP;
```

```
SELECT hire_date
FROM employees;
```

	HIRE_DATE
1	21-JUN-99 12.00.00.000000000 AM
2	13-JAN-00 12.00.00.000000000 AM
3	17-SEP-87 12.00.00.000000000 AM
4	17-FEB-96 12.00.00.000000000 AM
5	17-AUG-97 12.00.00.000000000 AM
6	07-JUN-94 12.00.00.000000000 AM
7	07-JUN-94 12.00.00.000000000 AM
8	07-JUN-94 12.00.00.000000000 AM

197

Exemple

```
CREATE TABLE web_orders
(order_date TIMESTAMP WITH TIME ZONE,
delivery_time TIMESTAMP WITH LOCAL TIME ZONE);
```

```
INSERT INTO web_orders values
(current_date, current_timestamp + 2);
```

```
SELECT * FROM web_orders;
```

	ORDER_DATE	DELIVERY_TIME
1	23-JUN-09 01.56.39.000000000 AM -05:00	25-JUN-09 01.56.39.000000000 AM

198

Types de données INTERVAL

Type INTERVAL : stocke la différence entre deux valeurs date-heure

2 catégories d'intervalle

- Année-mois
- Jour-heure

Précision de l'intervalle

- Le sous-ensemble de champs réel qui constitue un intervalle
- Définie dans le qualificatif d'intervalle

Type de données	Champs
INTERVAL YEAR TO MONTH	Year, Month
INTERVAL DAY TO SECOND	Days, Hour, Minute, Second with fractional seconds

199

Champs INTERVAL

Champ INTERVAL	Valeurs valides pour l'intervalle
YEAR	Tout entier positif ou négatif
MONTH	00 à 11
DAY	Tout entier positif ou négatif
HOURL	00 à 23
MINUTE	00 à 59
SECOND	00 à 59.9(N), où 9(N) est la précision

200

INTERVAL YEAR TO MONTH : Exemple

```
CREATE TABLE warranty (prod_id number, warranty_time INTERVAL YEAR(3) TO MONTH);
```

```
INSERT INTO warranty VALUES (123, INTERVAL '8' MONTH);
```

```
INSERT INTO warranty VALUES (155, INTERVAL '200' YEAR(3));
```

```
INSERT INTO warranty VALUES (678, '200-11');
```

```
SELECT * FROM warranty;
```

	PROD_ID	WARRANTY_TIME
1	123	0-8
2	155	200-0
3	678	200-11

201

Type DATE : Exemples

INTERVAL YEAR [(year_precision)] TO MONTH

INTERVAL '123-2' YEAR(3) TO MONTH

- Un intervalle de 123 ans, 2 mois

INTERVAL '123' YEAR(3)

- Un intervalle de 123 ans, 0 mois

INTERVAL '300' MONTH(3)

- Un intervalle de 300 mois

INTERVAL '123' YEAR

- Erreur, car la précision par défaut vaut 2 et '123' est sur 3 car.

202

INTERVAL DAY TO SECOND : Exemple

```
CREATE TABLE lab ( exp_id number, test_time INTERVAL DAY(2) TO SECOND);
```

```
INSERT INTO lab VALUES (100012, '90 00:00:00');
```

```
INSERT INTO lab VALUES (56098, INTERVAL '6 03:30:16' DAY TO SECOND);
```

```
SELECT * FROM lab; →
```

	EXP_ID	TEST_TIME
1	100012	90 00:00:00
2	56098	6 3:30:16.0

203

EXTRACT

Affichez la composante YEAR de SYSDATE.

```
SELECT EXTRACT (YEAR FROM SYSDATE) FROM DUAL;
```

EXTRACT(YEARFROMSYSDATE)
1 2009

Affichez la composante MONTH de HIRE_DATE pour les employés dont la valeur MANAGER_ID est 100.

```
SELECT last_name, hire_date, EXTRACT (MONTH FROM HIRE_DATE)
```

```
FROM employees
```

```
WHERE manager_id = 100;
```

	LAST_NAME	HIRE_DATE	EXTRACT(MONTHFROMHIRE_DATE)
1	Hartstein	17-FEB-1996 00:00:00	2
2	Kochhar	21-SEP-1989 00:00:00	9
3	De Haan	13-JAN-1993 00:00:00	1
4	Raphaely	07-DEC-1994 00:00:00	12
5	Weiss	18-JUL-1996 00:00:00	7

204

TZ_OFFSET

Affichez le décalage horaire correspondant aux fuseaux horaires 'US/Eastern', 'Canada/Yukon' et 'Europe/London' :

```
SELECT TZ_OFFSET('US/Eastern'),
```

```
       TZ_OFFSET('Canada/Yukon'),
```

```
       TZ_OFFSET('Europe/London')
```

```
FROM DUAL;
```

	TZ_OFFSET('US/EASTERN')	TZ_OFFSET('CANADA/YUKON')	TZ_OFFSET('EUROPE/LONDON')
1	-04:00	-07:00	+01:00

205

FROM_TZ

Affichez la valeur TIMESTAMP '2000-03-28 08:00:00' en tant que valeur TIMESTAMP WITH TIME ZONE pour le fuseau horaire 'Australia/North'

```
SELECT FROM_TZ(TIMESTAMP '2000-07-12 08:00:00',
```

```
       'Australia/North')
```

```
FROM DUAL;
```

FROM_TZ(TIMESTAMP'2000-07-1208:00:00','AUSTRALIA/NORTH')
1 12-JUL-00 08.00.00.000000000 AM AUSTRALIA/NORTH

206

TO_TIMESTAMP

Affichez la chaîne de caractères '2007-03-06 11:00:00'
en tant que valeur TIMESTAMP

```
SELECT TO_TIMESTAMP ('2007-03-06 11:00:00',
                     'YYYY-MM-DD HH:MI:SS')
FROM DUAL;
```

TO_TIMESTAMP('2007-03-06 11:00:00','YYYY-MM-DDHH:MI:SS')
06-MAR-07 11.00.00.000000000

207

TO_YMINTERVAL

Affichez une date située un an et deux mois après la date d'embauche des employés qui
travaillent dans le département dont la valeur DEPARTMENT_ID est égale à 20.

```
SELECT hire_date,
       hire_date + TO_YMINTERVAL('01-02') AS
       HIRE_DATE_YMININTERVAL
FROM   employees
WHERE  department_id = 20;
```

	HIRE_DATE	HIRE_DATE_YMININTERVAL
1	17-FEB-1996 00:00:00	17-APR-1997 00:00:00
2	17-AUG-1997 00:00:00	17-OCT-1998 00:00:00

208

TO_DSINTERVAL

Affichez une date située 100 jours et 10 heures après la date d'embauche de tous les employés.

```
SELECT last_name,
       TO_CHAR(hire_date, 'mm-dd-yy:hh:mi:ss') hire_date,
       TO_CHAR(hire_date +
               TO_DSINTERVAL('100 10:00:00'),'mm-dd-yy:hh:mi:ss') hiredate2
FROM   employees;
```

	LAST_NAME	HIRE_DATE	HIREDATE2
1	OConnell	06-21-99:12:00:00	09-29-99:10:00:00
2	Grant	01-13-00:12:00:00	04-22-00:10:00:00
3	Whalen	09-17-87:12:00:00	12-26-87:10:00:00
4	Hartstein	02-17-96:12:00:00	05-27-96:10:00:00
5	Fay	08-17-97:12:00:00	11-25-97:10:00:00
6	Mavris	06-07-94:12:00:00	09-15-94:10:00:00
7	Baer	06-07-94:12:00:00	09-15-94:10:00:00
8	Higgins	06-07-94:12:00:00	09-15-94:10:00:00

209

Type DATE : Exemples

INTERVAL DAY TO SECOND stocke une période de temps en termes de jours,
heures, minutes, & seconds.

INTERVAL DAY [(day_precision)] TO SECOND [(fractional_seconds_precision)]

INTERVAL '4 5:12:10.222' DAY TO SECOND(3)

- Indique 4 jours, 5 heures, 12 minutes, 10 secondes, and 222 centièmes de seconde

INTERVAL '7' DAY

- Indique 7 jours

INTERVAL '180' DAY(3)

- Indique 180 jours.

210

Type DATE : Exemples

INTERVAL '4 5:12' DAY TO MINUTE

- Indique 4 jours, 5 heures et 12 minutes.

INTERVAL '400 5' DAY(3) TO HOUR

- Indique 400 jours 5 heures.

INTERVAL '11:12:10.2222222' HOUR TO SECOND(7)

- Indique 11 heures, 12 minutes, and 10.2222222 secondes.

211

EFFECTUER UNE EXTRACTION HIÉRARCHIQUE

212

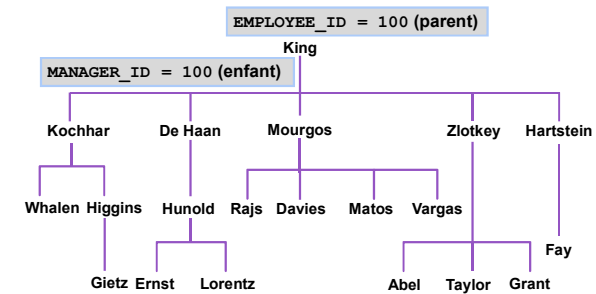
Echantillon de données table EMPLOYEES

	EMPLOYEE_ID	LAST_NAME	JOB_ID	MANAGER_ID
1	100	King	AD_PRES	(null)
2	101	Kochhar	AD_VP	100
3	102	De Haan	AD_VP	100
4	103	Hunold	IT_PROG	102
5	104	Ernst	IT_PROG	103
6	107	Lorentz	IT_PROG	103

16	200	Whalen	AD_ASST	101
17	201	Hartstein	MK_MAN	100
18	202	Fay	MK_REP	201
19	205	Higgins	AC_MGR	101
20	206	Gietz	AC_ACCOUNT	205

213

Arborescence naturelle



214

Interrogations hiérarchiques

```
SELECT [LEVEL], column, expr...  
FROM   table  
[WHERE condition(s)]  
[START WITH condition(s)]  
[CONNECT BY PRIOR condition(s)] ;
```

Avec condition : *expr comparison_operator expr*

215

Parcourir l'arborescence

Point de départ

- Indique la condition qui doit être satisfaite
- Accepte n'importe quelle condition valide

```
START WITH column1 = value
```

Avec la table EMPLOYEES, commencez par l'employé dont le nom est Kochhar

```
...START WITH last_name = 'Kochhar'
```

216

Parcourir l'arborescence

```
CONNECT BY PRIOR column1 = column2
```

Parcourez l'arborescence de haut en bas, à l'aide de la table EMPLOYEES

```
... CONNECT BY PRIOR employee_id = manager_id
```

Direction

De haut en bas ———→ Column1 = clé parent
Column2 = clé enfant

De bas en haut ———→ Column1 = clé enfant
Column2 = clé parent

217

Parcourir l'arborescence : De bas en haut

```
SELECT employee_id, last_name, job_id, manager_id  
FROM   EMPLOYEES  
START WITH employee_id = 101  
CONNECT BY PRIOR manager_id = employee_id ;
```

	1	2	3	4
	EMPLOYEE_ID	LAST_NAME	JOB_ID	MANAGER_ID
1	101	Kochhar	AD_VP	100
2	100	King	AD_PRES	(null)

218

Parcourir l'arborescence : de haut en bas

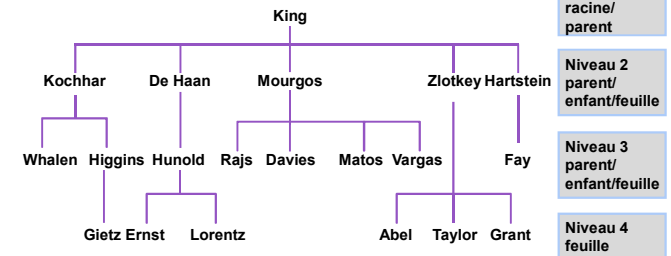
```
SELECT last_name||' reports to '||
PRIOR last_name "Walk Top Down"
FROM employees
START WITH last_name = 'King'
CONNECT BY PRIOR employee_id = manager_id ;
```

Walk Top Down
1 King reports to
2 King reports to
3 Kochhar reports to King
4 Greenberg reports to Kochhar
5 Faviel reports to Greenberg

105 Grant reports to Zlotkey
106 Johnson reports to Zlotkey
107 Hartstein reports to King
108 Fay reports to Hartstein

219

Classer des lignes avec la pseudo-colonne LEVEL



220

Formater des états hiérarchiques avec LEVEL et LPAD

Exemple Oracle :

Créer un état affichant les niveaux de direction de l'entreprise, en commençant par le niveau le plus élevé et en appliquant une indentation pour chaque niveau suivant.

```
SELECT LPAD(last_name, LENGTH(last_name)+(LEVEL*2)-2, '_')
AS org_chart
FROM hr.employees
START WITH first_name='Steven' AND last_name='King'
CONNECT BY PRIOR employee_id=manager_id
```

King
Kochhar
Greenberg
Faviel
Chen
Sciarra
Urman
Popp
Whalen
Mavris
Baer
Higgins
Gietz
De Haan
Hunold
Ernst
Austin

221

Eliminer des branches

Utiliser la clause WHERE pour éliminer un noeud

→ WHERE last_name != 'Higgins'

King
Kochhar
Greenberg
Faviel
Chen
Sciarra
Urman
Popp
Whalen
Mavris
Baer
Higgins
Gietz
De Haan
Hunold
Ernst
Austin

Utiliser la clause CONNECT BY pour éliminer une branche

→ CONNECT BY PRIOR

employee_id = manager_id
AND last_name != 'Higgins'



King
Kochhar
Greenberg
Faviel
Chen
Sciarra
Urman
Popp
Whalen
Mavris
Baer
De Haan
Hunold
Ernst
Austin
Pataballa
Lorentz
Raphaely

222

Présentation données

223

Opérateur de concaténation

Permet de :

- Concaténer des colonnes ou des chaînes de caractères avec d'autres colonnes sous la forme d'une nouvelle chaîne de caractères
- Syntaxe : ||

Exemple

```
SELECT last_name || job_id AS "Employees"  
FROM employees;
```

224

Littéraux

Une chaîne de caractères ou une date peut être présente dans un SELECT si elle est entre simples quotes

Les nombres sont affichés sans nécessité de reformatage

225

Délimitation avec (q)

Permet de définir son propre délimiteur

Exemple :

```
SELECT Attribut1 || q'[ Chaîne_delimiteur ]' || Attribut2 AS  
"EnteteDeColonne"
```

226

Variables de substitution

Permet de paramétrer les requêtes (WHERE, ORDER BY, SELECT complet, Noms de table) avec des valeurs temporaires saisie par l'utilisateur

Syntaxe : & ou &&

227

Exemple 1

```
SELECT Att1, Att2, Att3, Att4
```

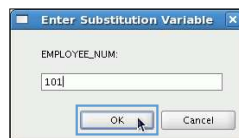
```
FROM R
```

```
WHERE Att4 = &NomVariable ;
```

→ Boîte de dialogue de saisie avec nom de la variable (ici EMPLOYEE_NUM)

228

Variable de substitution avec &



	EMPLOYEE_ID	LAST_NAME	SALARY	DEPARTMENT_ID
1	101	Kochhar	17000	90

229

Substitution

Noms de colonne, expressions et texte

Syntaxe :

```
SELECT liste_attributs, &NomAttribut
```

```
FROM Table
```

```
WHERE &condition
```

```
ORDER BY &attribut
```

230

Substitution

Pour la saisie des chaînes de caractères et des dates :

- Rajouter des apostrophes : Syntaxe :

```
SELECT...  
FROM...  
WHERE Att4='&NomVariable'
```
- Rajouter des apostrophes à la saisie

231

Substitution

avec && si réutilisation de la valeur de la variable

Syntaxe :

```
SELECT Liste_Attribut, &&NomVariable  
FROM table  
ORDER &NomVariable
```

232

Utilisation de variable “environnement”

DEFINE : création et affectation d’une valeur à une variable

UNDEFINE : suppression d’une variable

Syntaxe :

```
DEFINE NomVariable  
SELECT Liste_Attribut  
FROM...  
WHERE Attribut1=&NomVariable
```

VERIFY : verification de la requête avec la valeur remplacée

233

Prise en charge des expressions régulières

234

Expressions régulières ?

Recherche et manipulation des chaînes de caractères avec des modèles particuliers

Un SELECT permet de vérifier des informations stockées dans la BD

Formée avec

- des métacaractères pour indiquer les algorithmes de recherche
- des littéraux pour les caractères qu'on recherche

235

Conditions & fonctions d'expr. régulière

Nom de fonction ou de condition	Description
REGEXP_LIKE	Semblable à l'opérateur <code>LIKE</code> , mais permet la mise en correspondance d'expressions régulières plutôt que la simple mise en correspondance de modèles (condition).
REGEXP_REPLACE	Permet la recherche d'un modèle d'expression régulière et son remplacement par une chaîne de substitution.
REGEXP_INSTR	Permet la recherche d'un modèle d'expression régulière dans une chaîne et le renvoi de la position de cette chaîne.
REGEXP_SUBSTR	Permet la recherche d'un modèle d'expression régulière dans une chaîne donnée et l'extraction de la sous-chaîne correspondante.
REGEXP_COUNT	Renvoie le nombre de correspondances d'un modèle dans une chaîne d'entrée.

236

Métacaractères

Caractères générique,

Nb d'occurrence d'un caractère

Répétition du caractère

...

237

Métacaractère

Exemple : l'expression régulière `^(f|ht)tps?:$` recherche les éléments suivants à partir du début de la chaîne :

- Les littéraux `f` ou `ht`
- Le littéral `t`
- Le littéral `p`, éventuellement suivi du littéral `s`
- Le littéral deux-points `:"` à la fin de la chaîne

238

Description des méta. (doc ORACLE)

Syntaxe	Description
.	Correspond à un caractère quelconque dans le jeu de caractères pris en charge, à l'exception de NULL
+	Correspond à une ou plusieurs occurrences
?	Correspond à zéro ou une occurrence
*	Correspond à un nombre quelconque d'occurrences (zéro ou plus) de la sous-expression précédente
{m}	Correspond exactement à <i>m</i> occurrences de l'expression précédente
{m, }	Correspond à <i>m</i> occurrences au moins de la sous-expression précédente
{m,n}	Correspond à <i>m</i> occurrences au moins, mais pas à plus de <i>n</i> occurrences de la sous-expression précédente
[...]	Correspond à n'importe quel caractère unique de la liste entre crochets
	Correspond à l'une des alternatives
(...)	Considère l'expression entre parenthèses comme une unité. La sous-expression peut être une chaîne de littéraux ou une expression complexe contenant des opérateurs.

239

Description des métacaractères

Syntaxe	Description
^	Correspond au début d'une chaîne
\$	Correspond à la fin d'une chaîne
\	Considère le métacaractère suivant de l'expression comme un littéral
\n	Correspond à la <i>n</i> ème (1–9) sous-expression précédente d'une quelconque expression entre parenthèses. Les parenthèses entraînent la mémorisation d'une expression ; une référence arrière (backreference) y est associée.
\d	Un chiffre
[:class:]	Correspond à n'importe quel caractère appartenant à la classe de caractères POSIX indiquée ex. [:upper:]
[^:class:]	Correspond à n'importe quel caractère unique <i>ne</i> figurant pas dans la liste entre crochets

240

Syntaxe

```
REGEXP_LIKE (source_char, pattern [,match_option])
```

```
REGEXP_COUNT (source_char, pattern [, position [, occurrence [, match_option]]])
```

```
REGEXP_REPLACE(source_char, pattern [,replacestr [, position [, occurrence [, match_option]]]])
```

241

Syntaxe

```
REGEXP_SUBSTR (source_char, pattern [, position [, occurrence [, match_option [, subexpr]]]])
```

```
REGEXP_INSTR (source_char, pattern [, position [, occurrence [, return_option [, match_option [, subexpr]]]])
```

Paramètres

- **Occurrence** : quelle occurrence du modèle est recherchée
- **return_option** :
 - 0 : Renvoie la position du 1er caractère de l'occurrence (par défaut)
 - 1 : Renvoie la position du caractère suivant l'occurrence
- **match_parameter** :
 - "c" : Mise en correspondance avec distinction Maj/Min (par défaut)
 - "i" : Mise en correspondance sans distinction Maj/Min
 - "n" : Autorise l'opérateur de correspondance avec n'importe quel caractère
 - "m" : Chaîne source sur plusieurs lignes

242

Paramètres

Occurrence : quelle occurrence du modèle est recherchée

return_option :

- 0 : Renvoie la position du 1er caractère de l'occurrence (par défaut)
- 1 : Renvoie la position du caractère suivant l'occurrence

match_parameter :

- "c" : Mise en correspondance avec distinction Maj/Min (par défaut)
- "i" : Mise en correspondance sans distinction Maj/Min
- "n" : Autorise l'opérateur de correspondance avec n'importe quel caractère
- "m" : Chaîne source sur plusieurs lignes

243

REGEXP_LIKE

REGEXP_LIKE (source_char, pattern [, match_parameter])

Exemple

```
SELECT first_name, last_name
FROM employees
WHERE REGEXP_LIKE (first_name, '^Ste(v|ph)(e|a)n$');
```

244

REGEXP_REPLACE

REGEXP_REPLACE (source_char, pattern [, replacestr [, position [, occurrence [, match_option]]]])

Exemple

```
SELECT REGEXP_REPLACE(phone_number, '\.', '-') AS phone FROM
employees;
```

245

REGEXP_INSTR

REGEXP_INSTR (source_char, pattern [, position [, occurrence [, return_option [, match_option]]]])

Exemple

```
SELECT street_address,
REGEXP_INSTR(street_address,'[[:alpha:]]') AS
First_Alpha_Position
FROM locations;
```

246

